

AutoRAG

Self-Improving RAG Platform — Explained Simply

A Beginner's Guide to Autonomous AI Quality Systems

Version 2.2 | May 2026

What We'll Cover Today

01

The Problem — Why RAG is Hard

5 pain points teams face today

02

What AutoRAG Is

The closed-loop autonomous platform

03

How It Works — 7 Agents

Each agent's role, explained simply

04

The Unified Score

One number to measure everything

05

Diagnosing Failures

5 failure types and their fixes

06

Safety & Governance

Who approves what — and why

07

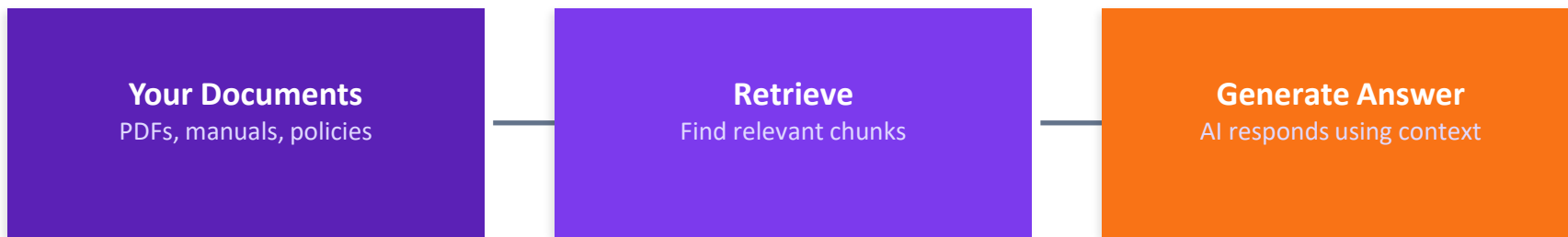
Rollout Plan & Business Case

Timeline, pricing, ROI

First — What is RAG?

RAG = Retrieval-Augmented Generation

A way to give AI assistants access to your company's documents — so they answer based on your data, not just general training.



⚠ The Challenge:

Building a RAG system is only the first step. Keeping it accurate, fast, and trustworthy over time — with no automated tools — is where teams get stuck.

5 Problems Teams Face with RAG Today

Manual Tuning Takes Days

Adjusting chunking, embeddings, and prompts requires 2–3 engineering days per iteration

No Clear Quality Score

Teams judge quality by gut feel and random sample queries — no reproducible standard

Every Change is Manual

Each configuration update requires a developer to re-run every experiment from scratch

No Production Quality Gate

Teams ship without knowing if the system hallucinates — users find out first

No Ground Truth Data

Labeling test datasets is expensive, so most teams skip evaluation entirely

So, What is AutoRAG?

AutoRAG is a platform that builds, tests, monitors, diagnoses, and improves your RAG system — automatically, with no manual work.

Closed Loop

Every pipeline is permanently watched. Problems are detected, diagnosed and fixed automatically.

Zero Labels Needed

AutoRAG generates its own test questions from your documents — no manual labeling required.

Enterprise Safe

Every change goes through a configurable approval gate. Nothing deploys without your rules.

How AutoRAG Works — 7 Specialised Agents

Each agent owns exactly one job. Together they form a continuous improvement loop.

1

Ingestion

Load & clean documents

2

Builder

Create the pipeline

3

Evaluator

Score quality (50 Q&As)

4

Observer

Watch for degradation

5

Diagnoser

Find root cause

6

Improver

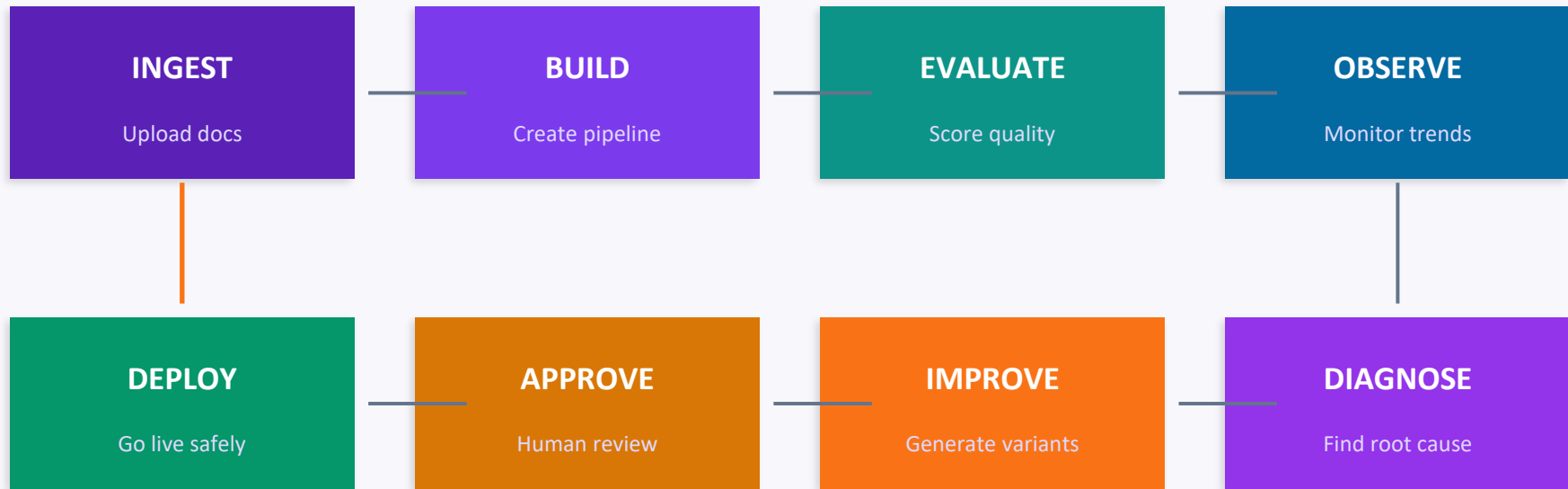
Generate better variants

7

Deployer

Deploy & rollback safely

The Closed-Loop — How It Never Stops Improving



The loop restarts automatically every time an improvement is approved and deployed.

The Unified Score — One Number That Measures Everything

$$\text{Score} = 0.25 \times \text{Retrieval} + 0.35 \times \text{Quality} + 0.25 \times \text{Faithfulness} - 0.10 \times \text{Latency} - 0.05 \times \text{Cost}$$

25%

Retrieval

Did we fetch the right chunks?

≥ 0.85 Recall@5

35%

Quality

Is the answer clear and complete?

≥ 0.80 (LLM judge)

25%

Faithfulness

Is it grounded in the documents?

≥ 0.80 (no hallucination)

-10%

Latency

How fast is the response?

p95 < 3 seconds

-5%

Cost

How much does a query cost?

< \$0.005 per query

≥ 0.85 → Production Ready

0.70 – 0.84 → Needs Improvement

< 0.70 → Below Threshold

5 Failure Types — AutoRAG Knows Exactly What Went Wrong



Retrieval Miss

Symptom:

Right answer exists in documents but wasn't found

Fix:

Increase top_k; adjust chunk overlap



Context Overflow

Symptom:

Too much context retrieved — AI gets confused by noise

Fix:

Reduce chunk size; add a reranker



Hallucination

Symptom:

Answer isn't grounded in documents (faithfulness < 0.5)

Fix:

Strengthen system prompt; add groundedness constraint



Answer Incomplete

Symptom:

Right docs found but key information missing from answer

Fix:

Adjust generation prompt; increase answer length target



Latency Spike

Symptom:

p95 query response time exceeds 5 seconds

Fix:

Switch to local embedding model; add caching

Pipeline Configurator — Set Up in Under 60 Seconds

No YAML. No code. Just 5 plain-English questions:

- 1 What is your domain?** → *Chunking strategy + domain-tuned prompt*
- 2 Who are your end users?** → *Answer style (technical vs. plain language)*
- 3 What is your biggest concern?** → *Score weight emphasis (faithfulness vs. speed)*
- 4 How should improvements be applied?** → *Approval mode (auto / human review / always review)*
- 5 Where will this be deployed?** → *Storage config + security defaults*

Output: Valid YAML Config + Scoring Profile + System Prompt + Plain Summary — all editable

Safety & Governance — Who Approves What

AutoRAG operates within human-defined risk boundaries. Every change is classified by risk level.

LOW RISK Auto-Approve	MEDIUM RISK Human-in-Loop	HIGH RISK Mandatory Gate
When it applies: Score improves ≥ 0.05; low-risk change (top_k, temperature, chunk_size) What happens: System deploys automatically. Change is logged to the audit trail.	When it applies: Enterprise mode enabled; or retrieval strategy is being swapped What happens: Approval request appears in your dashboard. You review the score difference and decide.	When it applies: AI model swap, system prompt change, or embedding model change What happens: Human approval ALWAYS required. Cannot be bypassed by any automation.

Every decision is written to an immutable audit log — compliance-ready from day one.

Who Uses AutoRAG?



Neha Sharma

ML Engineer · Fintech

Pain: Spends 3 days per iteration just to know if a config improved quality

✓ **Ships in one session with a reproducible Unified Score**



Rohan Mehta

Data Scientist · Research

Pain: Comparing 20 configurations takes a week of manual spreadsheet work

✓ **Side-by-side score comparison across all runs in minutes**



Priya Iyer

Enterprise Team Lead · Healthcare

Pain: Needs every change formally reviewed, logged, and rollback-ready

✓ **Mandatory Gate approval + one-click rollback in seconds**



Siddharth Rao

Indie Developer · SaaS Founder

Pain: Solo founder with no time to learn HNSW indexing or YAML tuning

✓ **5-question Configurator → production endpoint in 60 seconds**

Pricing Tiers & Enterprise ROI

Open Source	Pro	Enterprise	Enterprise Cloud
Free	₹3L / year	₹12–25L / year	₹25–60L / year
Founders & researchers	Small teams (≤ 10 engineers)	Mid-market & large enterprise	Managed SaaS (Phase 3)

Year 1 ROI — Enterprise Customer

Cost Centre	Without AutoRAG	With AutoRAG	Annual Saving
ML engineer tuning time	₹40 Lakh	₹6 Lakh	₹34 Lakh
Labeling / annotation	₹8 Lakh	₹0	₹8 Lakh
Production incidents	₹6 Lakh	₹1 Lakh	₹5 Lakh
AutoRAG license	—	₹15 Lakh	(₹15 Lakh)
Net Year 1 Saving	₹54 Lakh	₹22 Lakh	₹32 Lakh

45-Day MVP Roadmap & Three Phases

Phase A — Foundations

Days 1–10

- Docker Compose stack & database
- Ingestion Agent (PDF, HTML, TXT)
- Builder Agent (embedding pipeline)
- RAG Runtime + 14 FastAPI endpoints

Phase B — Quality Loop

Days 10–28

- Evaluator + Observer Agents
- Diagnoser Agent (5 failure types)
- Improver + Configurator + Approval
- Rollback Engine + MLflow integration

Phase C — Dashboard & Launch

Days 28–45

- Next.js dashboard (20 screens)
- Full test coverage & security audit
- Documentation + Quick Start guide
- v1.0.0 release → 26 June 2026

MVP Definition of Done: End-to-end pipeline live · Unified Score ≥ 0.80 · All 14 API endpoints · Dashboard navigable

AutoRAG — Key Takeaways



Closed-loop system that never needs manual tuning — improves itself automatically



One Unified Score (0.0–1.0) makes quality objective and comparable across teams



5 failure types tell engineers exactly what to fix — not just that something broke



Enterprise-safe: every high-risk change requires human approval; everything is audit-logged



60-second Pipeline Configurator — any user can set up a domain-tuned RAG pipeline



₹32 Lakh average Year 1 net saving per enterprise customer after licence cost